DocIndex

Lokalno indeksiranje i pretraživanje dokumenata

Matej Golub | Programiranje: Skriptni jezici | 30.10.2026.

Plan projekta za I. kontrolnu točku
https://github.com/matejgolub/DocIndex

1 Cilj i opseg

Za projekt sam odabrao izradu alata DocIndex u Pythonu. Alat će pretraživati sadržaj dokumenata spremljenih na računalu. Primjer primjene je mapa s materijalima za više kolegija: korisnik upiše pojam, a alat vrati dokumente u kojima se on pojavljuje, stranicu i kratak isječak teksta. Tako se traženi sadržaj može pronaći bez otvaranja svake datoteke zasebno.

Podržani formati bit će PDF, TXT i Markdown. PDF mora sadržavati tekst koji se može izdvojiti; prepoznavanje teksta sa skeniranih stranica nije predviđeno. Alat će raditi kroz terminal i spremati indeks u lokalnu SQLite bazu. Za rad neće trebati poslužitelj ni internetska veza. Izvorne dokumente samo će čitati.

2 Način rada i naredbe

Korisnik prvo zadaje mapu s dokumentima. Program prolazi kroz njezine podmape, izdvaja tekst podržanih datoteka i gradi indeks za kasniju pretragu. U bazi se čuvaju putanja, format, podaci o sadržaju i položaji riječi. Kod PDF-a tekst se obrađuje po stranicama kako bi rezultat vodio na pravo mjesto u dokumentu.

Naredba	Namjena
index <mapa>	Učitavanje dokumenata iz odabrane mape i izgradnja indeksa.
search <upit>	Pretraživanje pojmova ili fraza i ispis rangiranih rezultata.
update <mapa>	Dodavanje novih, ponovna obrada izmijenjenih i uklanjanje zapisa obrisanih datoteka.
stats	Broj dokumenata i riječi u indeksu te pregled zastupljenih formata.
report	Izrada HTML pregleda zbirke, vremena obrade i pogrešaka pri učitavanju.

Pretraga će imati filtar prema formatu i ograničenje broja rezultata. Uz rezultat će se ispisati putanja, ocjena podudaranja i okolni tekst s označenim pojmovima. Primjer planirane naredbe je python docindex.py search "baza podataka" --type pdf --limit 5. Točna fraza zadavat će se navodnicima unutar upita, primjerice search '"baza podataka"'.

3 Indeksiranje i pretraživanje

Jezgra projekta bit će vlastiti obrnuti indeks. Za svaku riječ pamti se u kojim se dokumentima pojavljuje i na kojim položajima. Upit se tako obrađuje preko indeksa, umjesto ponovnog čitanja svih datoteka. Isti postupak obrade primjenjuje se na dokument i upit: ujednačavanje Unicode zapisa, pretvaranje u mala slova i razdvajanje na riječi. Hrvatski znakovi č, ć, đ, š i ž ostaju sačuvani.

Za fraze će se provjeravati uzastopni položaji riječi. Upit s operatorom AND vratit će dokumente koji sadrže sve tražene pojmove, a OR dokumente koji sadrže barem jedan. Početna verzija tražit će zapisani oblik riječi, bez svođenja različitih padeža na isti oblik.

4 Rangiranje i osvježavanje indeksa

Rezultate ću rangirati vlastitom implementacijom TF-IDF izračuna i kosinusne sličnosti. TF opisuje koliko je riječ zastupljena u dokumentu, a IDF smanjuje važnost riječi koje se pojavljuju u velikom dijelu zbirke. Time dokument koji često spominje traženi stručni pojam može dobiti prednost pred dokumentom u kojem se taj pojam javlja samo usput. Kosinusna sličnost uspoređivat će težine pojmova u upitu i dokumentu, uz normalizaciju duljine vektora.

Primjerice, za upit „relacijska baza" očekivano je da bilješke o relacijskim bazama budu ispred opće skripte iz programiranja. Baza će čuvati podatke potrebne za ponovni izračun težina nakon promjene zbirke. SQLite služi za pohranu; obrnuti indeks, spajanje rezultata upita i rangiranje bit će napisani u Pythonu.

Kod naredbe update uspoređivat će se SHA-256 sažeci sadržaja. Tekst i položaji riječi ponovno će se izdvajati samo za nove ili izmijenjene datoteke. Za obrisanu datoteku uklonit će se i njezini zapisi iz indeksa, a zatim osvježiti statistike zbirke koje utječu na rangiranje. Tako rezultati neće upućivati na dokumente kojih više nema.

5 Tehnologije i struktura koda

Koristit ću Python 3.11 ili noviji. Za terminalsko sučelje predviđen je argparse, za putanje pathlib, za bazu sqlite3, a za sažetke hashlib. Tekst PDF dokumenata izdvajat će biblioteka pypdf. TXT i Markdown datoteke čitat će se kao običan tekst; Markdown oznake neće se posebno prikazivati. Za provjeru pojedinih dijelova koristit ću pytest.

Kod će biti podijeljen prema zadatku: cli.py za naredbe, readers/ za čitanje formata, index.py za tokenizaciju i indeks, search.py za upite i rangiranje, storage.py za bazu te report.py za HTML izvješće. U tests/ nalazit će se testovi i mali primjeri dokumenata s poznatim sadržajem. Ovakva podjela omogućuje zasebnu provjeru čitanja, pretraživanja i pohrane.

6 Plan razvoja

Razdoblje	Zadaci	Rezultat
Do 27.11.	Čitanje PDF, TXT i MD datoteka, obrada teksta, SQLite i osnovni indeks.	Dokumenti se učitavaju i mogu pretraživati po riječi.
Do 18.12.	TF-IDF, rangiranje, fraze, AND i OR, filtri i isječci teksta.	Upit vraća poredane rezultate s mjestom podudaranja.
Do 22.01.	Ažuriranje zbirke, statistika, HTML izvješće, testovi i mjerenje brzine.	Dovršen alat s uputama i pripremljenim primjerom rada.

7 Testiranje i mogući problemi

Za testiranje ću pripremiti zbirku s unaprijed poznatim sadržajem. Provjerit ću poredak rezultata, fraze, AND i OR upite, hrvatske znakove te PDF s više stranica. Nakon izmjene, dodavanja ili brisanja datoteke provjerit ću odgovara li indeks novom stanju. Brzinu pretrage usporedit ću s izravnim pretraživanjem izdvojenog teksta iste zbirke, uz odvojeno mjerenje početnog indeksiranja.

Najveći problem očekujem kod PDF dokumenata s više stupaca ili neispravnim redoslijedom teksta. Takvi primjeri bit će uključeni u provjeru fraza i prikaza isječaka. Nečitljiva ili zaštićena datoteka zabilježit će se kao pogreška, a obrada ostalih nastavit će se. Za veće zbirke dokumenti će se obrađivati pojedinačno kako se cijeli sadržaj ne bi morao držati u memoriji.